

RATE BEER



Pedro
Azevedo
47094

Aprendizagem Automática
LEIM – 21-01-2024

ÍNDICE

Introdução

Dados

Construção do Vocabulário

Metodologias de Teste e Métricas de Desempenho

Classificação Binária

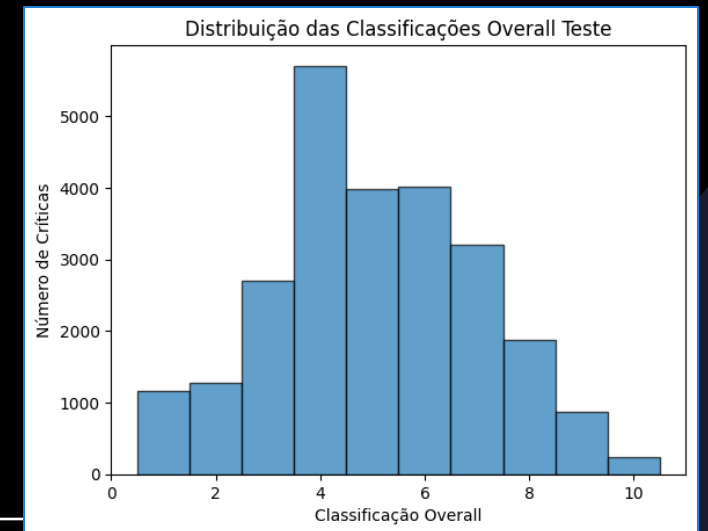
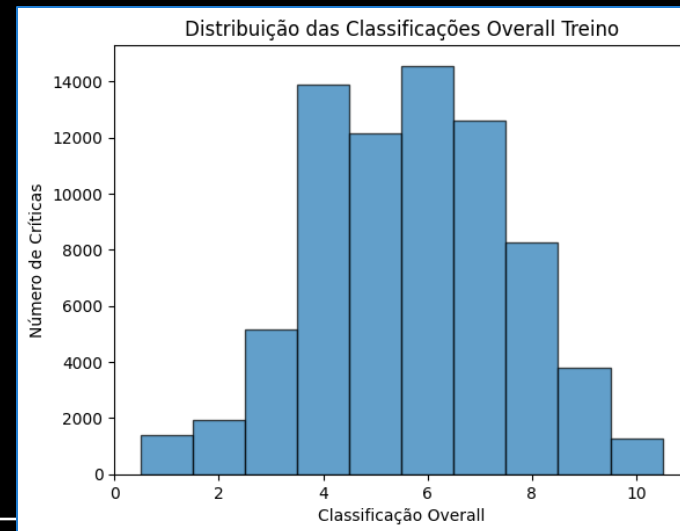
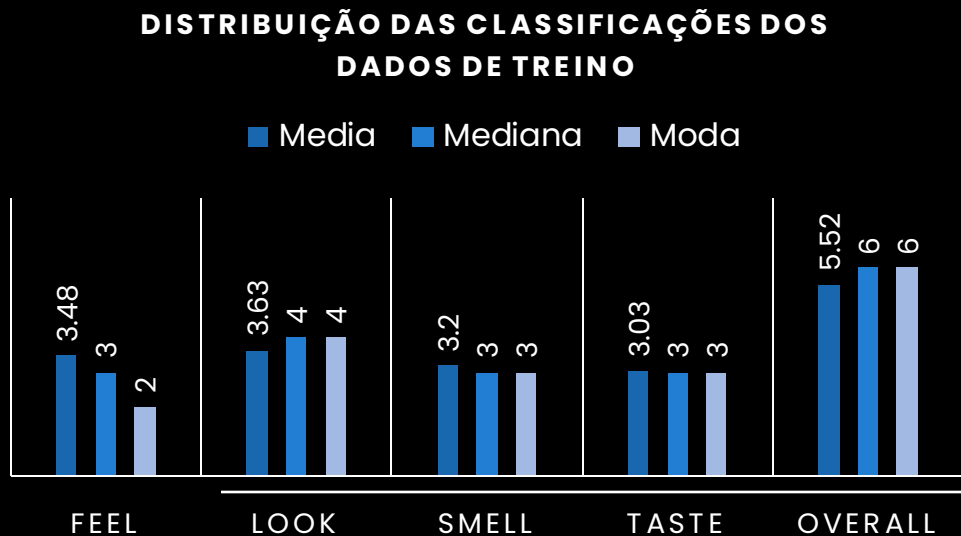
Classificação Multi-classe

PCA

Conclusões

1. Introdução e Dados

- Neste trabalho de aprendizagem automática, foram fornecidos dados de críticas de cerveja constituído pela classificação do *smell*, *taste*, *look*, *feel* e uma classificação *overall*, sendo esta ultima numa escala de 1 a 10 e as restantes de 1 a 5. Adicionalmente, tem uma *review* em texto.
- O objetivo do trabalho é determinar a qualidade de uma cerveja baseado no que foi escrito sobre a mesma.
- Começou-se por analisar os dados, vendo aqui alguns valores estatísticos das classificações dos diferentes componentes
- Nos dois histogramas estão representadas o número de críticas por cada valor de classificação global, do lado esquerdo temos os dados de treino e do lado direito os dados de teste.



2. Construção do Vocabulário

Para a construção do vocabulário, iniciou-se por limpar os documentos.

Retirou-se as *tags* de HTML '
'

Eliminou-se todos os caracteres que não sejam alfabéticos, através de expressões regulares, substituindo os caracteres por um espaço em branco

De seguida testou-se a utilização de *stemmers* para a redução das palavras ao seu prefixo, sendo estes:

- *PorterStemmer*
- *SnowballSteamer*
- *LancasterStemmer*

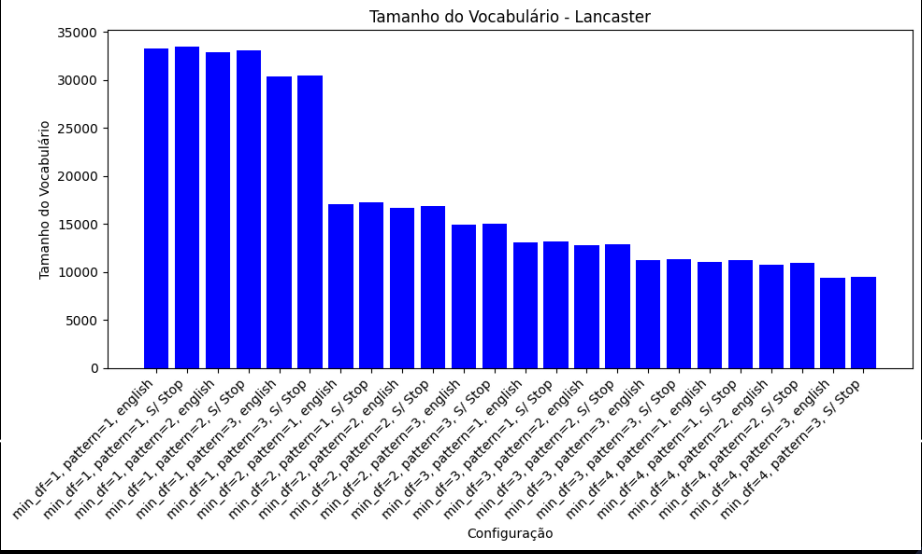
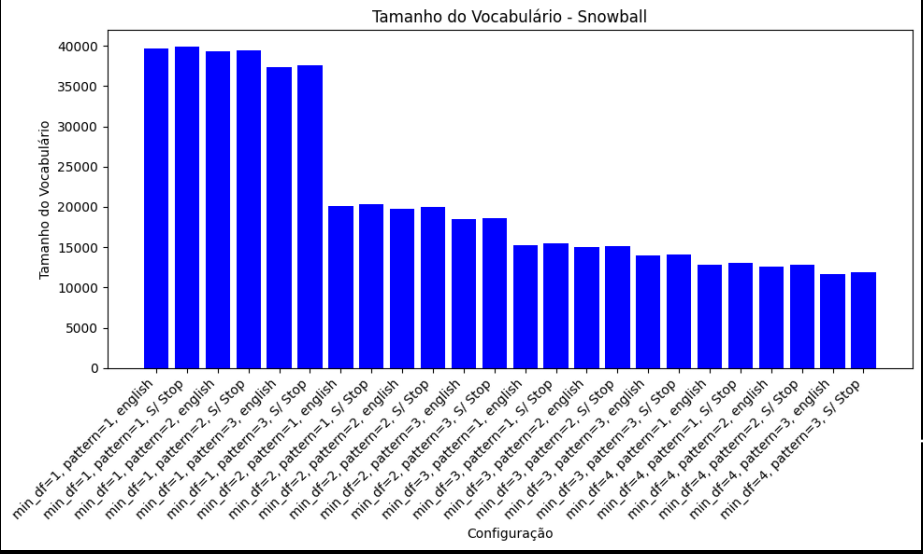
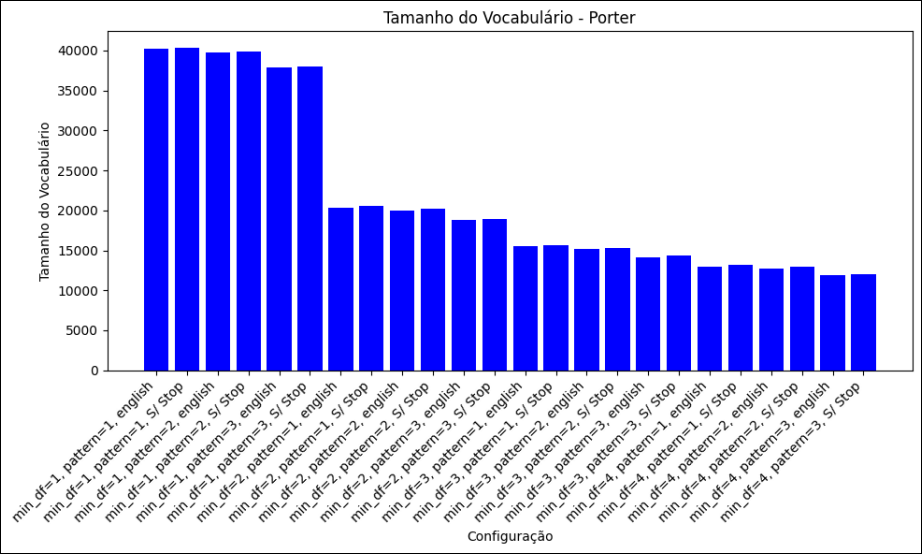
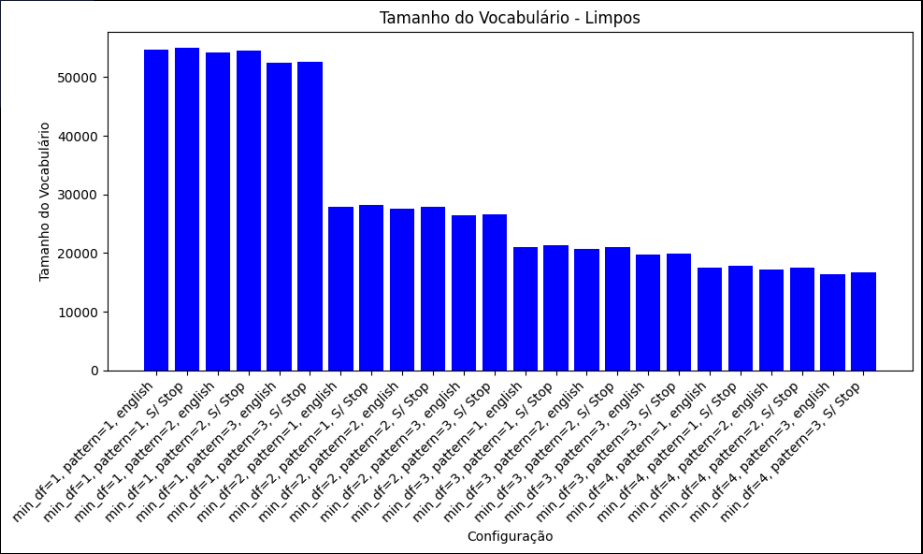
Tf-idf – Vectorizer

É aplicada a técnica que representa a matriz de contagens documento-termo, dando relevância a termos mais informativos, verificando a frequência da palavra no documento e no corpus.

Aqui existem certos parâmetros que vão afetar a atribuição de importância, que vão ser testados. Estes são:

- *min_df* – ignorar palavras que aparecem em menos de *n* documentos;
- *token_pattern* – especifica uma expressão regular, e vai extrair palavras com *n* caracteres;
- *stop-words* – retira as palavras mais frequentes de uma língua, estas não tendo significado.
- *n-gramas* – que indica o numero de palavras que se coloca na mesma célula

- Para iniciar a decisão de parâmetros, aplicou-se os *stemmers* e os parâmetros para vectorização, gerando estas tabelas onde se pode ver o tamanho do vocabulário para as diferentes configurações. Podendo concluir que não faz sentido adicionar ao vocabulário palavras que apenas aparecem uma vez no *corpus*, logo aí existe uma grande redução do tamanho. Mais a frente vai se testar os diferentes vocabulários com o Discriminante Logístico para escolher a melhor configuração.



3. Metodologias de Teste e Métricas de Desempenho

- Através das tabelas anteriores, não é possível saber qual o vocabulário, e as configurações que vão beneficiar na classificação. Assim sendo é necessário testar com um classificador. E para cada tipo de classificação, pode ser benéfico usar diferentes configurações. Assim já se volta a pegar nos vocabulários.
- Um método de teste é utilizar o *cross validation* onde se divide o conjunto de treino em k folds, e alternadamente cada um vai servir como conjunto de validação e os restantes de treino, fazendo uma média dos scores.
- Para encontrar estas configurações, podemos verificar diferentes métricas:
 - Matriz de Confusão – que permite ver como foram feitas as classificações e quão assertivas, sendo que se pode calcular ainda Precisão, Recall, F1-score.
 - Curva de ROC – possibilita a visualização dos resultados para todos os limiares possíveis.

4. Classificação Binária

- Para a classificação binária, vai se dividir os dados em duas classes (1, -1), e classifica-se como cerveja **Muito Boa** (*overall* ≥ 9) e **Muito Má** (*overall* ≤ 2), e foram aplicadas duas metodologias:
 - 1ª- Considerou-se todas as criticas, sendo que se fez duas classificações, a primeira para a Muito Boa, onde a classe 1 corresponde a cervejas com *overall* maior ou igual a 9 e as restantes pertencem a classe -1. E uma segunda para Muito Má, onde a classe 1 corresponde a criticas menores ou iguais a 2, e -1 as restantes.
 - 2ª - Considerando apenas as criticas Muito Boas e Muito Más, 1 e -1, respetivamente.
-

1º Classificador Binário – Discriminante Logístico – 1ª Abordagem

- Tem como objetivo estimar a matriz de peso W , que vai ser utilizada para classificar os dados de teste. Aqui os pesos vão ser atribuídos as palavras e assim é possível classificar a crítica com base nesses valores.
- Aqui é usada a função do *sklearn*, *LogisticRegression*, esta tendo hiper-parâmetros que podem ser ajustados para uma melhor classificação.
- Mas primeiro tem que se verificar qual o melhor *stemmer* a utilizar para a limpeza e qual a melhor configuração para a vetorização.

Testou se os seguintes valores para a vetorização para cada *stemmer* aplicando a função *LogisticRegression()*:

- *ngrams* = [(1,1), (1,2), (1,3)]
- *mdfs* = [2,3,4,5]
- *tps* = [r'\b\w\w+\b', r'\b\w\w\w+\b', r'\b\w\w\w+\b']

Para cada *stemmer* verificou-se os valores das diferentes combinações dos valores de precisão, *recall*, *f1-score*, e os *scores* de treino e de validação. Podendo-se os gráficos no próximo *slide*.

Os valores de *recall* e de *f1-score*, são todos muito baixos, e os *scores* todos altos, sem grande variação, não estando presente nesta apresentação os gráficos, sendo que não influenciaram a escolha.

- Analisando os parâmetros, retirou-se uma das configuração de cada *stemmer* com melhores valores no Discriminante Logístico, tendo em conta que não deve haver sobre aprendizagem, ou seja, os *scores* de treino e de validação não devem ser 1 nem muito diferenciados, haver um precisão ou *recall* a 1. Retirou-se então as seguintes:

Stemmer	N-grammas	Precisão	Recall	F1-score	Score treino	SValidação
Limpos	min_df=5, pattern=1, (1, 3)	0.851064	0.0401606	0.0767018	0.941829	0.942221
Porter	min_df=4, pattern=2, (1, 2)	0.830508	0.0491968	0.092891	0.942639	0.942581
Snowball	min_df=4, pattern=2, (1, 3)	0.833333	0.0401606	0.0766284	0.942099	0.942161
Lancaster	min_df=4, pattern=1, (1, 2)	0.84	0.0421687	0.0803059	0.942281	0.942281

- Obtendo por utilizar o Stemmer Porter, tendo um F1 mais equilibrado, com a configuração:

min_df=4, pattern=2, n-gramas (1, 2) Stop_Words='English'

Procurar as melhores hiper-parâmetros do modelo

Na função do `LogisticRegression()`, existem parâmetros que influenciam a classificação, e os que se vão testar são os seguintes:

- *Penalty* – A regularização para minorar a sobre aprendizagem sendo que *l1* a *Lasso* e a *l2* a *Ridge*;
- *C* – controla a regularização, sendo o inverso do peso, quanto maior o valor menor a regularização. Utilizando os seguintes valores para teste: 0.1, 1, 10, 50, 100, 150, 200

Após treinar e testar o modelo com as varias combinações, verificou-se para o melhor valor de cada métrica, qual os parâmetros que resultam esses valores, para analisar as diferenças, e decidir, qual a métrica a que melhor se ajusta ao problema. Estes foram os resultados:

Métricas	Valor	C	Penalty		
Score Cross-Validation	0.9497	0.1	l1		
Erros	1258	0.1	l1		
Score do Modelo	0.9497	0.1	l1		
Distancia Prec/Rec	0.8860	1.0	l2	Prec: 0.146	Rec: 0.7512
Distancia ROC	0.3068	1.0	l2	FPR: 0.270	TPR: 0.837

Havendo apenas duas configurações, pode-se testar o Discriminante com ambas e comparar melhor todas as métricas.

C	Penalty	Score CV	Erros	Score	Dist Prec/Rec	Precisão	Recall	Dist Roc	FPR	TPR
0.1	l1	0.9497	1258	0.9497	0.9359	0.0693	0.9014	0.4606	0.2979	0.6486
1.0	l2	0.9495	1264	0.9494	0.8659	0.1690	0.7568	0.3068	0.2390	0.8076

Em termos de scores do *cross validation* e de erros, os valores são muito semelhantes, contudo, nas distancias das curvas, já se nota uma diferença, e como o melhor valor é o que tem a menor distancia, a configuração com C=1 e penalty=l2 é preferida.

Resultados do Discriminante Logístico - Binário

Treinou-se e testou-se o modelo com os parâmetros e todos os dados de treino e teste, efetuando para cada classificação, Muito Boa e Muito Má, tendo os seguintes resultados:

Classificar	Erros	Score	Score Treino	Precisão	Recall	F1	T positivos	T negativos	F positivos	F Negativos
Muito Boa	868	0.9653	0.9509	0.9995	0.9657	0.9823	24122	10	856	12
Muito Má	2407	0.9037	0.9466	0.9974	0.9045	0.9487	22269	324	2350	57

Comparando o score do treino com o do modelo, a partida o modelo não entra em sobre-aprendizagem, sendo que os valores são muito próximos. Contudo, o valor de precisão é 0.99, o que também pode ser um indicativo de *overfitting*. Verifica-se também que existe sempre um numero alto de Verdadeiros positivos, classificando bem as criticas como não Muito Boas ou não Muito Más. Contudo existe muitos erros a classificar as criticas que são na verdade Muito Boas ou Muito Más, tendo um grande numero de Falsos positivos e menor Verdadeiros Negativos, sendo que ser positivo é o valor de -1 e o negativo 1.

Olhando para uma critica de 9 valores que não foi bem classificada, esta é mais descritiva, não fornecendo palavras com pesos significantes:

- Critica: "You might notice that Louisiana beers (dixi, acadiana) tend to have a cider taste to them somehow. I dont know if they keep them in cider barrels or what, but Abita typicaly doesnt have that distinction. This particular seasonal abita beer (after fall fest and before red ale) does have that distinction and is particularly good during christmas time. Its aftertaste tastes like you drank out of a can, but the initial taste makes up for that fault. I prefer to drink it with something sweet like a desert, but it is definatly not something I would like to drink all day. Do give it a shot."

Concluindo, este classificador, é um mau classificador.

2º Classificador Binário – Máquina de Vetores de Suporte – 1ª Abordagem

O segundo classificador binário a testar é a SVM. Aqui é escolhida uma fronteira de decisão que maximiza a margem entre as classe, ou seja, a distancia entre a fronteira de decisão e os pontos mais próximos de cada classe.

Aqui também foram testados parâmetros do *C*, o *Kernel*, que especifica o tipo de função usada para transformar os dados de entrada, e o *Gamma*, que controla a complexidade do modelo, sendo que um valor mais alto resulta de uma menor complexidade. Os parâmetros vão ser testados com os seguintes valores: *Gamma*: 0.01, 0.1, 1; *C*: 0.1, 1, 10; *Kernel*: 'linear', 'rbf'.

Resultados do SVM – Binário

Comparou-se os scores dos modelos para obter os melhores parâmetros fazendo varias combinações de parametros. Este processo demorou 8 horas, e não foi utilizado o *Cross Validation*, onde poderia ser mais assertivo nos parâmetros a utilizar, contudo despendia de mais tempo.

Os parâmetros que resultaram foi o *Kernel rbf*, *Gamma* de 1, e *C* de 10. Porém os valores após re-treinar, indicaram sobre aprendizagem: Score treino: 0.9998, Score: 0.94, Precisão: 0.9976, Recall: 0.9435;

Apos averiguar outros parâmetros, e após vários testes de alternativas, todos continham uma precisão de 1, podendo ver um exemplo de todas as métricas: Parametros escolhidos: kernel=rbf, gamma=0.01, C=10

score de treino	score de teste	erros	precisão	recall	F1	Tpos	Tneg	Fpos	Fneg
0.93836	0.89304	2674	1.0	0.89304	0.9434	22326	0	2674	0

E como se pode ver, é um resultado muito mau sendo que classifica tudo como sendo da classe -1, a critica não é Muito Má, neste caso. Sendo que este processo demorou 1:30 horas, não se vai testar mais parâmetros, pois o tempo de espera é muito grande e já fora testados vários com os resultados semelhantes.

2ª Abordagem na Classificação Binária

Uma segunda abordagem na classificação binária consistiu em ignorar as criticas que não eram nem Muito Boas nem Muito Más. Assim sendo tinha se as duas classes representadas respetivamente por 1 e -1.

Executando o procedimento aplicado na 1ª abordagem, chegou-se a conclusão que a melhor forma de limpar os documentos e vetorizar é com o *Lancaster Stemmer*, *min_df* de 4, palavras com 2 ou mais caracteres, e uni grama.

Procurando os melhores parâmetros para o *LogisticRegression()*, tem se um *C* de 1 e *penalty l2*

Também se realizou esta abordagem para o segundo classificador, SVM, onde os parâmetros escolhidos foram o *Kernel rbf*, *gamma* de 0.01 e *C* de 10.

Aqui pode-se observar os resultados para cada classificador através da segunda abordagem.

	Erros	Score	S-Treino	Precisão	Recall	F1	T positivos	T negativos	F positivos	F Negativos	Pt Operacional
SVM	205	0.9421	0.9571	0.9510	0.9641	0.9575	2310	1025	86	119	(0.049,0.923)
Discriminante	215	0.9393	0.9623	0.9473	0.9635	0.9553	2301	1024	87	128	(0.084,0.958)

Os valores já são bastante melhores em relação á 1ª abordagem.

Os resultados entre os dois classificadores são bastante semelhantes, contudo o SVM tem os melhores, com mais Verdadeiros Positivos e Verdadeiros negativos, ou seja compara bem as criticas Muito Más e Muito Boas. O ponto operacional mais a esquerda é também no SVM, que é o ideal.

Ambos os classificadores são aceitáveis.

5. Classificador Multi-classe – Discriminante Logístico

Para a este problema de classificação pretende-se prever a pontuação de três aspetos das criticas, o *smell*, *taste*, *overall*.

Utilizou-se o Discriminante Logístico, onde se procurou de novo a melhor configuração e *stemmer* para limpar e criar o vocabulário, onde pelo mesmo processo da classificação binária se decidiu utilizar os seguinte parâmetros:

Snowball, *min_df=5* *pattern=4*, *ngramas (1,1)*

Testou-se também os melhores parâmetros através da componente do *smell*, desta vez adicionando o parâmetro *Solver*, que pode ser *lbfgs*, *saga*, *sag*. Onde resultou os seguinte parâmetros:

Saga, *penalty l1*, *C = 1*

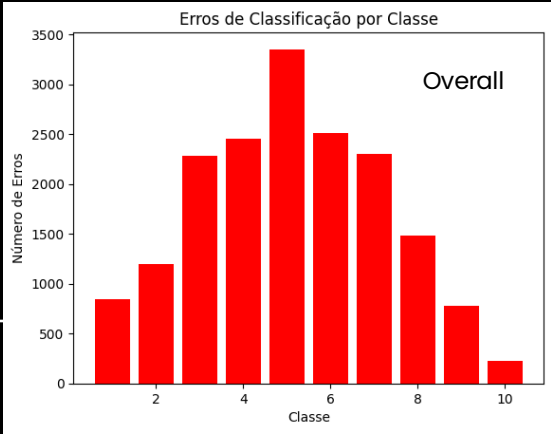
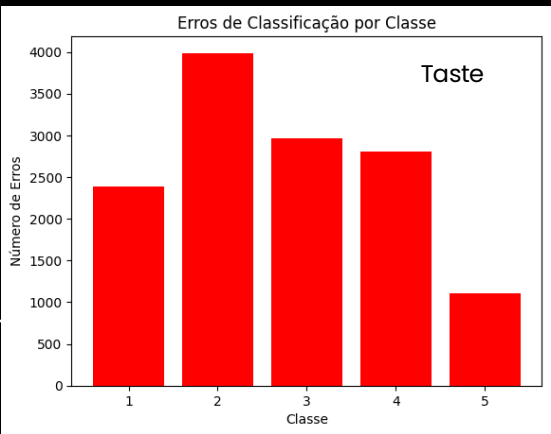
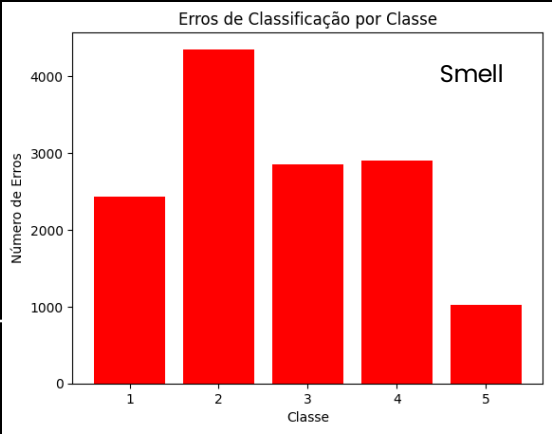
Resultados do Discriminante Logístico – Multi-classe

Após o treino e a classificação do modelo com os dados todos temos os seguinte valores na tabela a esquerda.

Podemos ver valores muito baixos em termos de *score*. Tendo uma percentagem de erros inferior a 50%. Podendo concluir que é um mau classificador.

É de notar que as classes com mais erros são as centrais onde as opiniões são mais neutras.

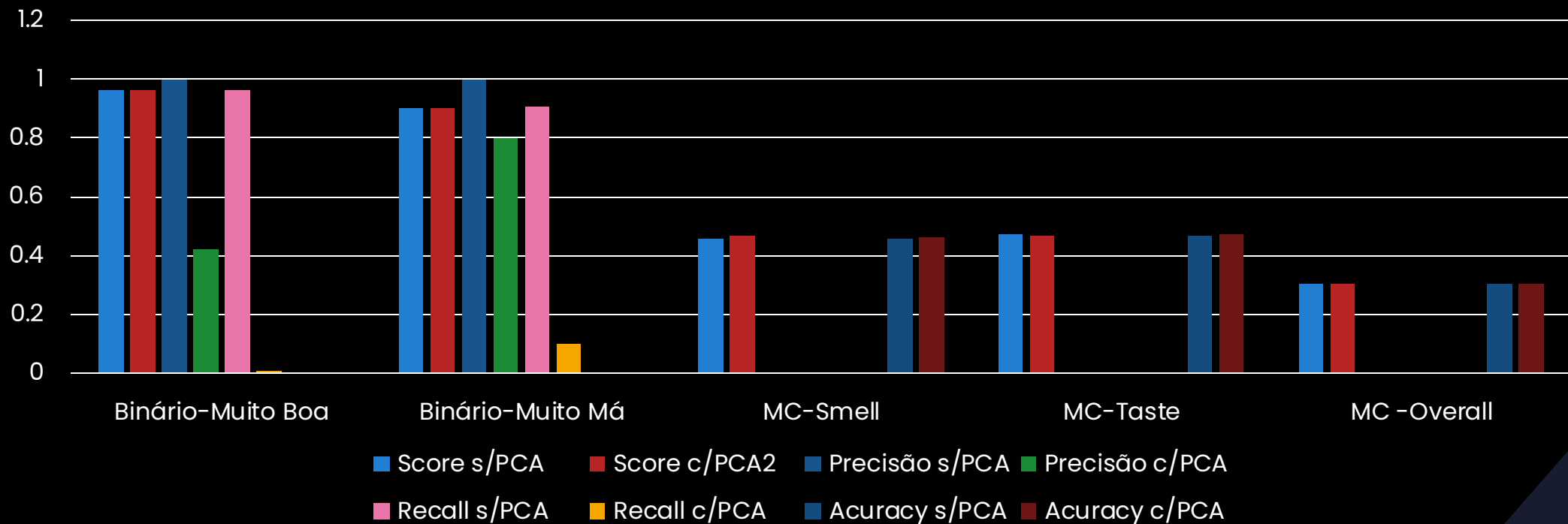
	Erros	Score	Score Treino	Acuracy
smell	13567	0.45732	0.51645	0.4574
taste	13253	0.46988	0.53029	0.4692
overall	17435	0.3026	0.3804	0.30296



6. PCA

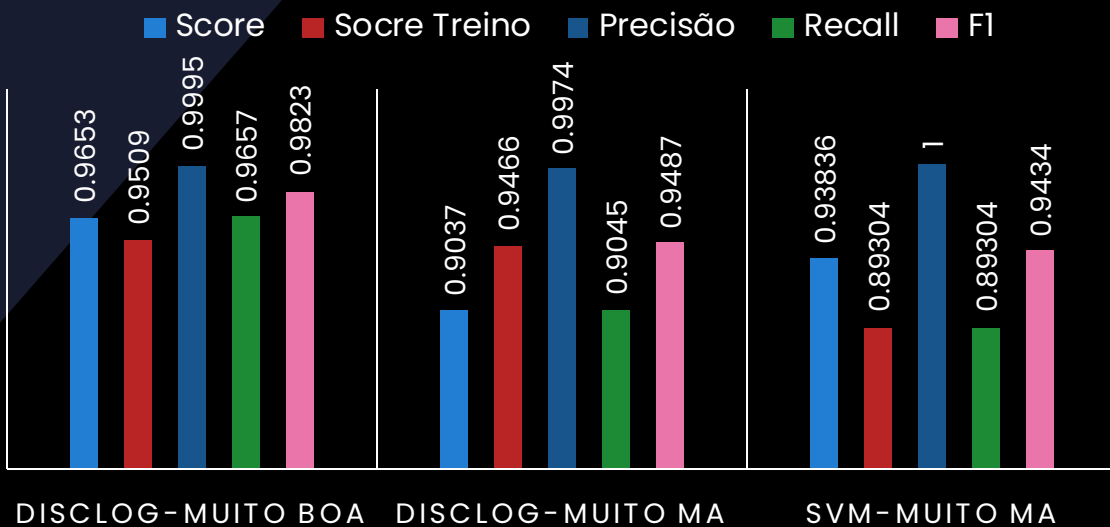
- O método PCA é não supervisionado, sendo que não existe classes, onde é aplicado a todas os dados, não só as classes. O objetivo de utilizar este método é reduzir a dimensão dos dados, tentando preservar o mais possível a variação presente nos dados de alta dimensão, sendo este um método de pré-processamento de dados.
 - Ao reduzir o número de variáveis pode-se tornar o processo de classificação mais rápido e preciso, sendo que ao reduzir o número de variáveis, reduz-se também o número de parâmetros a serem treinados, o que pode levar a um melhor desempenho do modelo. Este método combina as variáveis originais em um conjunto de menor componentes principal, combinações lineares das variáveis, sendo estes componentes escolhidos de forma a maximizar a variância total nos dados.
 - Neste problema, onde estamos a utilizar dados de texto para classificar críticas, está-se a utilizar matrizes esparsas, onde o PCA pode não ser eficiente. Assim sendo, tem que se utilizar e testar o *Truncated SVD* (*Singular Value Decomposition*), que é uma técnica relacionada com o PCA, mas eficiente para com matrizes esparsas. Neste problema, quando é criado o vocabulário, tem-se uma matriz onde cada linha é um documento e cada coluna é as palavras do vocabulário com os valores correspondentes a frequência no documento, sendo que a maioria das palavras vai estar a 0, tornando esta matriz esparsa, ou seja cada crítica é representada por um vetor de grande dimensão, e o *Truncated SVD* reduz essa dimensão ficando com as mais importantes, sendo que se cria um vetor com a direção no espaço onde há mais variação.
-

Concluindo, no caso binário, não é benéfico usar PCA, a não ser que os valores melhorem com aumentando o número de componentes, tendo em conta que requer mais poder computacional. No caso multi-classe, nota-se pequenos melhoramentos, e com o aumento das componentes vai ter valores melhores.

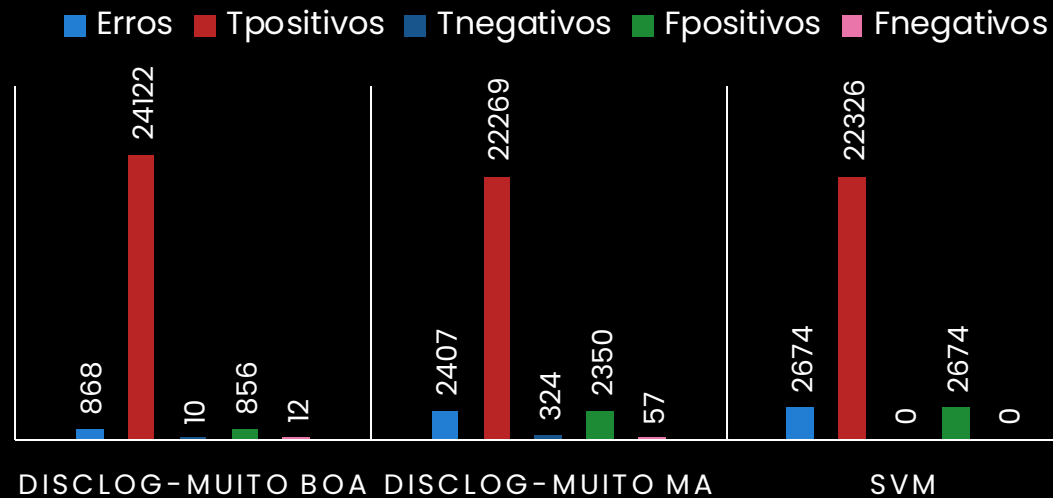


7. Conclusões e reunião de resultados

CLASSIFICAÇÃO BINÁRIA



CLASSIFICAÇÃO BINARIA



Nenhum dos classificadores binários são bons classificadores, observado os valores tem-se muitos Verdadeiros positivos (-1) assim como Falsos negativos, sendo que quando se está a classificar como muito boa, falha a maioria das vezes, tal como quando se classifica como muito má.

Um dos motivos para o sucedido, é o de só se estar a considerar cerveja muito boa com um valor maior ou igual a 9, sendo que cervejas acima de 5 valores podem ter palavras com sentido positivo presentes nas *reviews* de 9 valores, o que no classificador vai dar um peso mais negativo. O mesmo acontece com as criticas muito más, mas no sentido contrário. Ignorando as criticas intermédias, como se fez na segunda abordagem já é possível ter uma boa classificação.

Um segundo motivo pelo qual as críticas podem ser classificadas de forma errada, é que a *review* pode não ter sentido positivo ou negativo, ser mais descritiva, ou haver apenas um aspecto que o crítico deu mais ênfase na *review*.

Relativamente ao classificador multi-classe, com valores de acerto abaixo de 0.5, são valores baixos, não sendo um bom classificador.

	Erros	Score	Score Treino	Acuracy
smell	13567	0.45732	0.51645	0.4574
taste	13253	0.46988	0.53029	0.4692
overall	17435	0.3026	0.3804	0.30296

Discriminante Logístico
Classificação Multi-classe